

# SECURE CODING REVIEW

Security Vulnerability Assessment Report

|                      |                                          |
|----------------------|------------------------------------------|
| Target Application   | Django Login / Signup Web Application    |
| Language & Framework | Python 3 / Django                        |
| Database             | SQLite3                                  |
| Review Type          | Manual Code Inspection + Static Analysis |
| Intern               | Laiba Arshad                             |
| Student ID           | CA/DF1/58052                             |
| Organization         | CodeAlpha Cybersecurity Internship       |
| Review Date          | May 2026                                 |
| Report Version       | v1.0                                     |

## SEVERITY CLASSIFICATION

|          |      |        |     |
|----------|------|--------|-----|
| CRITICAL | HIGH | MEDIUM | LOW |
|----------|------|--------|-----|

# 1. EXECUTIVE SUMMARY

This report presents the findings of a comprehensive secure code review conducted on a Django-based Login and Signup web application. The review was performed as part of the CodeAlpha Cybersecurity Internship program (Task 3: Secure Coding Review). The audit involved manual code inspection and static analysis techniques to identify security vulnerabilities, insecure coding patterns, and areas of improvement.

The application provides user authentication functionality including registration (signup) and login features using Django's built-in ORM with SQLite3 database. During the review, several vulnerabilities were identified ranging from critical injection risks to medium-severity configuration issues.

| Severity | Count | Status                 |
|----------|-------|------------------------|
| CRITICAL | 2     | Requires Immediate Fix |
| HIGH     | 2     | Fix Before Deployment  |
| MEDIUM   | 2     | Fix Soon               |
| LOW      | 2     | Best Practice          |
| TOTAL    | 8     | Vulnerabilities Found  |

## 2. VULNERABILITY FINDINGS

### VULN-01

### SQL Injection via Raw Query

CRITICAL

**CWE Reference:** [CWE-89: Improper Neutralization of Special Elements in SQL Command](#)

The application uses raw SQL queries with direct string formatting to authenticate users. This allows attackers to manipulate SQL logic by injecting malicious input, potentially bypassing authentication or dumping the entire database. For example, entering ' OR '1'='1 as username would log in without a valid password.

#### ■ Vulnerable Code:

```
# views.py – VULNERABLE
def login_view(request):
    username = request.POST['username']
    password = request.POST['password']
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    cursor.execute(query) # DANGEROUS – raw string formatting!
```

#### ■ Secure Fix:

```
# views.py – SECURE FIX
from django.contrib.auth import authenticate, login
def login_view(request):
    username = request.POST['username']
    password = request.POST['password']
    user = authenticate(request, username=username, password=password)
    if user: # Django ORM handles parameterized queries safely
        login(request, user)
```

**Recommendation:** Always use Django's ORM (`authenticate()`, `User.objects.get()`) or parameterized queries. Never format user input directly into SQL strings.

### VULN-02

### Passwords Stored in Plaintext

CRITICAL

**CWE Reference:** [CWE-256: Plaintext Storage of a Password](#)

User passwords are stored directly in the database without hashing. If the database is compromised, all user passwords are immediately exposed. This is a fundamental security failure — passwords must always be stored as cryptographic hashes.

#### ■ Vulnerable Code:

```
# models.py – VULNERABLE
class User(models.Model):
    username = models.CharField(max_length=100)
    password = models.CharField(max_length=100) # Storing raw password!

# views.py
User.objects.create(username=username, password=password) # NO hashing!
```

#### ■ Secure Fix:

```
# SECURE FIX – Use Django's built-in User model
from django.contrib.auth.models import User
from django.contrib.auth.hashers import make_password

# Django automatically hashes passwords with PBKDF2
user = User.objects.create_user(username=username, password=password)
user.save() # Password is hashed automatically – never stored in plaintext
```

**Recommendation:** Use Django's built-in User model which automatically applies PBKDF2 hashing with salt. Never store raw passwords. If using custom model, use `make_password()` from `django.contrib.auth.hashers`.

**VULN-03****Missing CSRF Protection on Forms****HIGH****CWE Reference: CWE-352: Cross-Site Request Forgery (CSRF)**

Login and signup forms do not include the {% csrf\_token %} template tag. Without CSRF tokens, attackers can craft malicious web pages that silently submit forms on behalf of authenticated users, potentially changing passwords, creating accounts, or performing unauthorized actions.

**■ Vulnerable Code:**

```
<!-- login.html - VULNERABLE -->
<form method='POST' action='/login/'>
  <!-- Missing {% csrf_token %} -->
  <input type='text' name='username'>
  <input type='password' name='password'>
  <button type='submit'>Login</button>
</form>
```

**■ Secure Fix:**

```
<!-- login.html - SECURE FIX -->
<form method='POST' action='/login/'>
  {% csrf_token %} <!-- Django generates a unique token per session -->
  <input type='text' name='username'>
  <input type='password' name='password'>
  <button type='submit'>Login</button>
</form>
```

**Recommendation:** Add {% csrf\_token %} inside EVERY POST form. Also ensure CsrfViewMiddleware is in MIDDLEWARE in settings.py (it is enabled by default in Django — do not remove it).

**VULN-04****DEBUG Mode Enabled in Production****HIGH****CWE Reference: CWE-489: Active Debug Code**

settings.py has DEBUG=True which should NEVER be used in a production environment. When DEBUG is True, Django displays full stack traces, source code snippets, local variable values, and database queries in the browser whenever an error occurs — leaking sensitive internal information to attackers.

**■ Vulnerable Code:**

```
# settings.py - VULNERABLE
DEBUG = True          # NEVER True in production!
ALLOWED_HOSTS = []    # Empty = accepts all hosts (dangerous)
SECRET_KEY = 'django-insecure-hardcoded-key-12345' # Hardcoded!
```

**■ Secure Fix:**

```
# settings.py - SECURE FIX
import os
DEBUG = False          # Always False in production
ALLOWED_HOSTS = ['yourdomain.com', 'www.yourdomain.com']
SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY') # Load from env variable
```

**Recommendation:** Set DEBUG=False before deploying. Use environment variables for SECRET\_KEY. Define ALLOWED\_HOSTS explicitly. Never commit SECRET\_KEY to version control (GitHub).

**VULN-05****No Brute Force / Rate Limiting Protection****MEDIUM****CWE Reference: CWE-307: Improper Restriction of Excessive Authentication Attempts**

The login endpoint has no limit on failed login attempts. An attacker can perform unlimited password guessing (brute force) or credential stuffing attacks without any lockout or delay mechanism. This can lead to account takeover given enough time.

■ **Vulnerable Code:**

```
# views.py – VULNERABLE
def login_view(request):
    # No attempt counter, no lockout, no CAPTCHA
    user = authenticate(username=username, password=password)
    if user:
        login(request, user)
    else:
        return HttpResponse('Invalid credentials') # Try again infinitely!
```

■ **Secure Fix:**

```
# SECURE FIX – Install django-axes
# pip install django-axes

# settings.py
INSTALLED_APPS += ['axes']
AXES_FAILURE_LIMIT = 5 # Lock after 5 failed attempts
AXES_COOLLOFF_TIME = 1 # Lockout for 1 hour
AUTHENTICATION_BACKENDS = ['axes.backends.AxesBackend', ...]
```

**Recommendation:** Use django-axes or django-ratelimit to limit failed login attempts. After 5 failures, lock the account temporarily. Consider adding CAPTCHA for extra protection.

VULN-06

Sensitive Data Exposed in URL Parameters

MEDIUM

**CWE Reference:** CWE-598: Information Exposure Through Query Strings in GET Request

Some views pass user credentials or session tokens as GET parameters in the URL. URLs are logged in server access logs, browser history, and Referer headers — exposing sensitive data to unintended parties including log monitoring systems and third parties.

■ **Vulnerable Code:**

```
# VULNERABLE – Credentials in URL
# URL: /login?username=laiba&password;=mypassword123

def login_view(request):
    username = request.GET.get('username') # GET = visible in URL!
    password = request.GET.get('password') # Stored in server logs!
```

■ **Secure Fix:**

```
# SECURE FIX – Always use POST for sensitive data
def login_view(request):
    if request.method == 'POST': # POST data not in URL
        username = request.POST.get('username')
        password = request.POST.get('password')
    else:
        return redirect('login') # Reject GET requests
```

**Recommendation:** Never pass passwords, tokens, or sensitive data via GET parameters. Always use POST method for forms that submit credentials or sensitive information.

VULN-07

Missing Input Validation & Sanitization

LOW

**CWE Reference:** CWE-20: Improper Input Validation

User input fields (username, password) have no server-side validation for length, character type, or format. This can lead to unexpected behavior, database errors, or XSS if input is later reflected in HTML without escaping.

■ **Vulnerable Code:**

```
# views.py – VULNERABLE
def signup_view(request):
    username = request.POST['username'] # No length check
    password = request.POST['password'] # No complexity check
    email = request.POST['email'] # No format validation
    User.objects.create(username=username, password=password)
```

■ **Secure Fix:**

```
# SECURE FIX – Use Django Forms with validators
from django import forms
from django.core.validators import MinLengthValidator

class SignupForm(forms.Form):
    username = forms.CharField(max_length=50, min_length=3)
    password = forms.CharField(min_length=8, widget=forms.PasswordInput)
    email = forms.EmailField() # Built-in email format validation
```

**Recommendation:** Use Django Forms or ModelForms for all user input. They provide built-in validation, sanitization, and error handling. Add password strength requirements.

**VULN-08**

**HTTPS Not Enforced (Missing Security Headers)**

**LOW**

**CWE Reference: CWE-319: Cleartext Transmission of Sensitive Information**

The application does not enforce HTTPS or set security-related HTTP headers. Without HTTPS, all data (including passwords) is transmitted in plaintext and can be intercepted via Man-in-the-Middle attacks. Missing security headers expose the app to clickjacking and MIME-type sniffing attacks.

■ **Vulnerable Code:**

```
# settings.py – MISSING security settings
# SECURE_SSL_REDIRECT not set
# SECURE_HSTS_SECONDS not set
# X_FRAME_OPTIONS not set
# SECURE_CONTENT_TYPE_NOSNIFF not set
```

■ **Secure Fix:**

```
# settings.py – SECURE FIX
SECURE_SSL_REDIRECT = True # Force HTTPS
SECURE_HSTS_SECONDS = 31536000 # HSTS for 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_CONTENT_TYPE_NOSNIFF = True # Prevent MIME sniffing
X_FRAME_OPTIONS = 'DENY' # Prevent Clickjacking
SESSION_COOKIE_SECURE = True # HTTPS-only cookies
CSRF_COOKIE_SECURE = True # HTTPS-only CSRF cookie
```

**Recommendation:** Enable all Django security settings before deployment. Use SSL/TLS certificate (free via Let's Encrypt). Run `python manage.py check --deploy` to see all security warnings.

### 3. OVERALL RECOMMENDATIONS

---

#### Immediate Priority (Fix Before Launch)

- Fix SQL injection by replacing raw queries with Django ORM.
- Enable password hashing using Django's built-in User model.
- Add {% csrf\_token %} to all POST forms immediately.

#### Before Deployment

- Set DEBUG=False and configure ALLOWED\_HOSTS in settings.py.
- Move SECRET\_KEY to environment variables — never commit to GitHub.
- Add brute force protection using django-axes library.

#### Best Practices

- Always use POST method for sensitive form submissions.
- Validate all user input using Django Forms or ModelForms.
- Enable all Django security settings (HTTPS, HSTS, security headers).
- Run python manage.py check --deploy before every production release.

## 4. TOOLS & METHODOLOGY

---

### Review Methodology:

- Manual Code Inspection — Line-by-line review of views.py, models.py, settings.py, and HTML templates.
- Static Analysis — Identifying insecure patterns without executing the code.
- OWASP Top 10 Checklist — Cross-referencing findings against the OWASP Top 10 Web Application Security Risks.
- CWE Mapping — Each vulnerability mapped to its Common Weakness Enumeration (CWE) identifier.
- Django Security Documentation — Referenced official Django security best practices guide.

### Tools Used:

| Tool                 | Purpose                                  |
|----------------------|------------------------------------------|
| Python 3             | Manual code analysis and scripting       |
| Django Debug Toolbar | Identifying query and performance issues |
| Bandit               | Python static analysis security linter   |
| OWASP ZAP            | Web application vulnerability scanner    |
| SQLMap (controlled)  | SQL injection vulnerability verification |
| Mozilla Observatory  | HTTP security header analysis            |

## 5. CONCLUSION

---

The security audit of the Django Login/Signup application revealed 8 vulnerabilities across 4 severity levels. The most critical issues are SQL Injection and Plaintext Password Storage, both of which must be resolved immediately as they pose direct risk of data breach and account compromise.

The good news is that Django provides excellent built-in security tools — using Django's ORM, built-in User model, CSRF middleware, and security settings resolves most of these issues with minimal code changes. Secure coding is not about writing complex code; it is about using the right tools correctly.

---

Prepared by: Laiba Arshad | CodeAlpha Cybersecurity Internship | Task 3: Secure Coding Review | May 2026